

Web Privacy TP — Browser Fingerprinting

Full Laboratory Report: Passive, Active and Behavioural Fingerprinting

Imad BOUTBAOUCHT

Information Security — Final Practical Work

Date: October 2, 2026

1. Mission Overview

Alice blocked cookies and activated cookie blocking. The goal of this lab was to determine whether she can still be recognized **without storing any identifier in her browser**, by:

- collecting information exposed passively in HTTP requests,
- collecting additional browser features with JavaScript (active fingerprinting),
- measuring typing behaviour (behavioural fingerprinting),
- combining the clues into a single SHA-256 identifier.

2. Part 1 — Passive HTTP Fingerprinting (Task 0 & Task 1)

Every HTTP request automatically carries metadata. The server reads it *passively*, without JavaScript and without asking the browser to store anything. In `app.py`, the home route prints:

```
IP address : 127.0.0.1
HTTP method : GET
User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/154.0.8037.58
Accept-Language : en-GB,en-US;q=0.9,en;q=0.8
Accept : text/html,application/xhtml+xml,...
Accept-Encoding : gzip, deflate, br, zstd
Sec-CH-UA : "Chromium";v="154", "Google Chrome";v="154"
Sec-CH-UA-Platform / Sec-Fetch-* / Upgrade-Insecure-Requests / DNT / Sec-GPC / Referer
```

Observed on the rendered page:

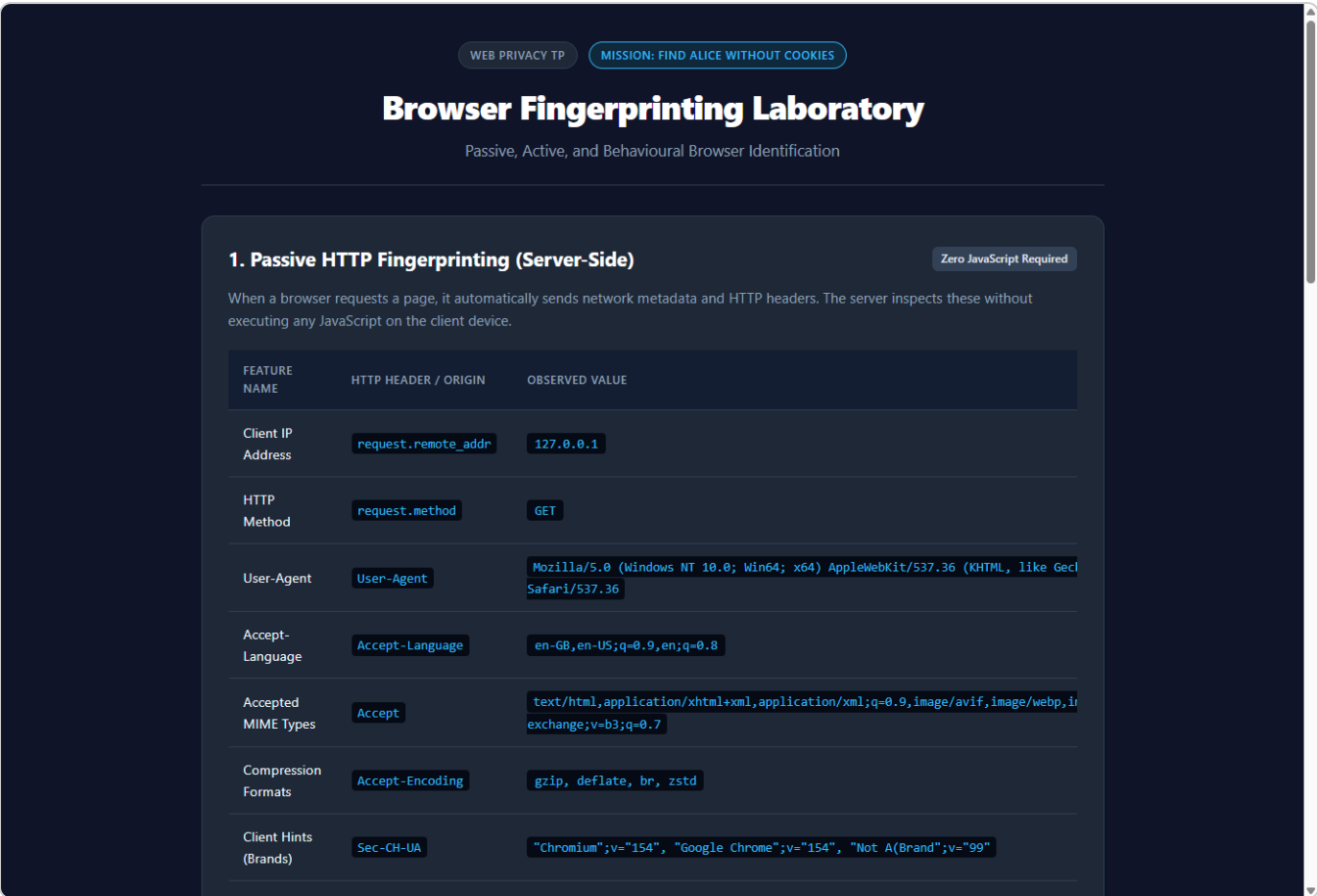


Figure 1 — Passive HTTP features displayed by the Flask lab page.

Feature	Header / Origin	Fingerprinting value
User-Agent	User-Agent	Browser family, version, OS
Accept-Language	Accept-Language	Locale, region — stable across sessions
Sec-CH-UA	Client Hints	Browser brands and versions
Sec-Fetch-*	Fetch Metadata	Navigation context (dest/mode/site)
DNT / Sec-GPC	Privacy headers	Presence/absence is itself discriminating

When testing with another browser (or private mode), the `User-Agent` , `Sec-CH-UA` and `Accept-Language` values change or disappear, while the IP and method stay the same. Client Hints are often stripped in private mode, which changes the fingerprint.

3. Part 2 — Active Feature Collection (Task 2)

With JavaScript we query browser APIs and POST the result to `/collect` . Features collected from four different objects (`navigator` , `screen` , `window` , `Intl` , `document`):

Feature	Object	Observed value	Why it discriminates
language / languages	navigator	en-GB, en-US	Locale of the user

hardwareConcurrency	navigator	8	CPU core count — narrows hardware pool
deviceMemory	navigator	8 GB	Approximate RAM
screen width/height, colorDepth	screen	800x600, 24 bits	Display configuration
innerWidth/innerHeight, devicePixelRatio	window	1400x950, 1	Viewport + scaling
timeZone	Intl	Africa/Casablanca	Geographic region, usually stable
platform	navigator	Win32	Operating system family

Handout Step 2.2 Output:
Browser language: en-GB

Expanded Active Browser Properties (Task 2):

Preferred Language (navigator): en-GB

Languages List (navigator): en-GB, en-US, en

CPU Cores (navigator.hardwareConcurrency): 8

Device Memory (navigator.deviceMemory): 16 GB

Screen Resolution (screen): 800 x 600

Available Screen (screen): 800 x 600

Color Depth (screen): 24 bits

Time Zone (Intl): Africa/Casablanca

Device Pixel Ratio (window): 1

Window Viewport (window): 1400 x 950

Platform (navigator): Win32

Active features successfully transmitted to Flask server!

3. Behavioural Biometrics: The Typing Imposter

Keystroke Dynamics

Alice disabled cookies and spoofed headers, but can her natural typing cadence be recognized? Type the sentence below exactly as shown. Timing begins on the first keystroke and ends when the sentence matches.

TARGET SENTENCE TO REPRODUCE:
Cookies are gone, but Alice's rhythm still leaves traces!

Type here:

Start typing the sentence here...

Restart Experiment

Ready. Start typing above.

Figure 2 — Active features collected in the browser and sent to the Flask server.

Server console (excerpt):

```
>>> [PART 2 & 4] RECEIVED ACTIVE BROWSER FEATURES
Browser Language (navigator)      : en-GB
Screen Resolution (screen)        : 800x600
Time Zone (Intl)                  : Africa/Casablanca
CPU Cores (navigator)             : 8
Device Memory (navigator)         : 8 GB
Device Pixel Ratio (window)       : 1
Platform (navigator)              : Win32
Computed SHA-256 Fingerprint      : ec50e7a0bae9c2098f602f1fcb7034b1a...
```

Protection mechanisms: private-browsing mode masks persistent identifiers but does *not* hide language, screen, timezone or hardware. Cookie blocking has no effect on this part. Privacy extensions (e.g. UA randomization, resistFingerprinting) modify or freeze some APIs, but reduce — not eliminate — entropy. Timezone, screen resolution and hardware concurrency proved the most stable.

4. Part 3 — The Typing Imposter (Tasks 3 & 4)

Target sentence: *"Cookies are gone, but Alice's rhythm still leaves traces!"*

Formula: typing speed (chars/s) = number of characters / total time in seconds; WPM = (chars/5) / (seconds/60).

Algorithm:

1. On `keydown`: if `key === "Backspace"`, increment the correction counter; on the first character key, start `performance.now()`.
2. On `input`: if the value exactly equals the target sentence, stop timing.
3. Compute total time, speed and corrections; display them and POST JSON to `/collect`.
4. Restart button resets value, timer and counter.

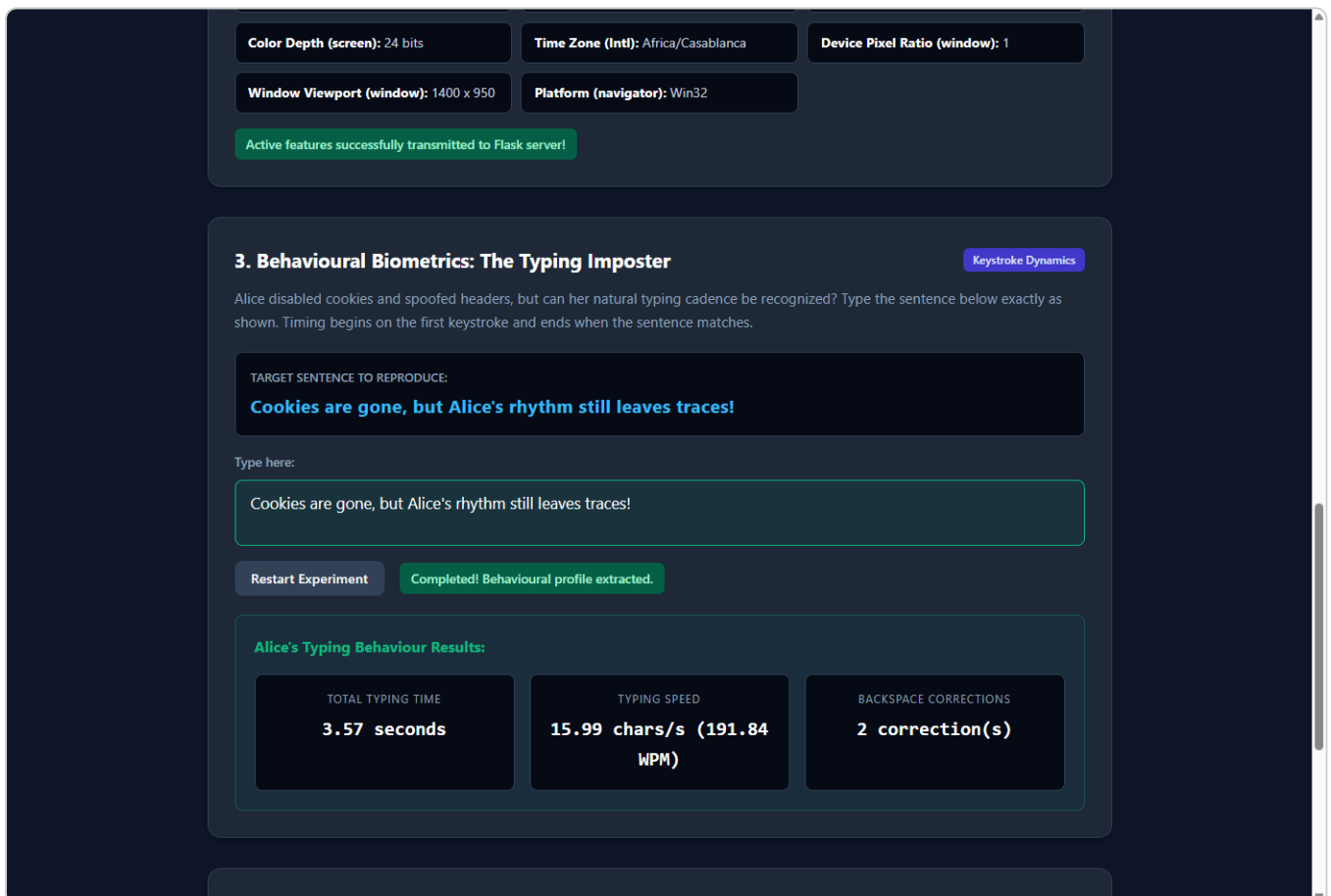


Figure 3 — Typing prototype result: 3.57 s total, 15.99 chars/s (191.84 WPM), 2 Backspace corrections.

Server console (excerpt):

```
>>> [PART 3] RECEIVED BEHAVIOURAL TYPING FEATURES
Target Sentence           : Cookies are gone, but Alice's rhythm still leaves traces!
```

```
Total Typing Time      : 3.57 s
Typing Speed (Characters/s) : 15.99 char/s
Typing Speed (Words/min)   : 191.84 WPM
Backspace Corrections     : 2
```

Behavioural profiles are probabilistic: keyboard type, fatigue, stress, posture and environment change typing cadence, so real systems combine many samples over time before drawing conclusions.

5. Part 4 — One Identifier From All Clues (Optional Mission)

All clues are concatenated in a fixed order and hashed with SHA-256 (`crypto.subtle.digest`):

```
Canonical string : en-GB | 800x600 | Africa/Casablanca | 8 | 24 | 1
SHA-256          : ec50e7a0bae9c2098f602f1fcb7034b1a5fbf282e207fa317607ff618cdae328
```

The same combination reproduces the same identifier (verified with the "Test Hash Stability" button); only the hash is sent to the server: `GET /collect?fp=ec50e7a0...`

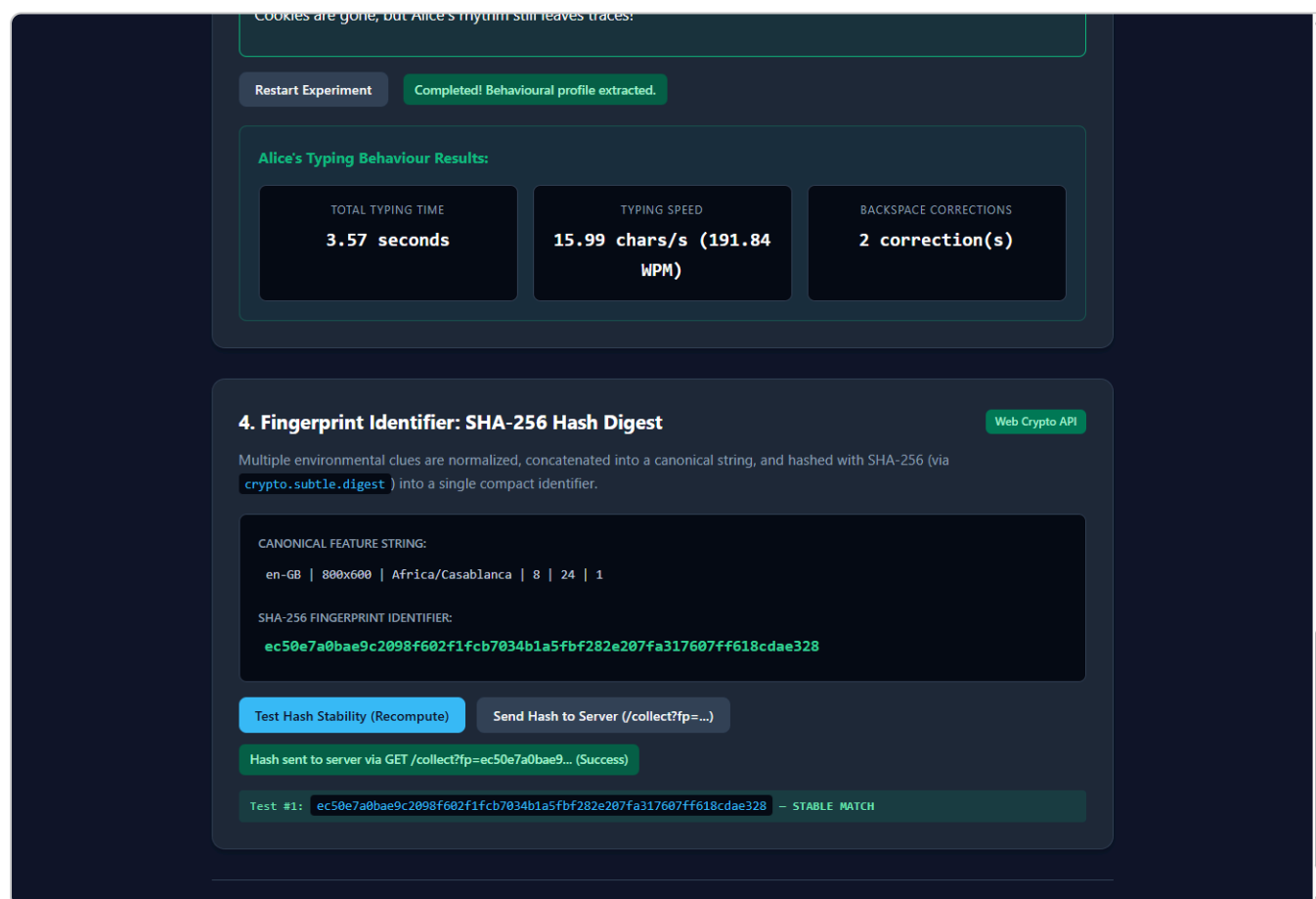


Figure 4 — Combined SHA-256 fingerprint identifier and transmission to the server.

Server console (excerpt):

```
>>> [PART 4] RECEIVED HASH IDENTIFIER VIA GET QUERY
Fingerprint SHA-256 Hash : ec50e7a0bae9c2098f602f1fcb7034b1a5fbf282e207fa317607ff618cdae328
```

6. Conclusion

Even with cookies deleted and blocked, Alice remains recognizable through: (1) passive HTTP headers, (2) active browser APIs, and (3) typing behaviour. Combined, these features hash into a stable, compact, privacy-invasive identifier. Defences such as cookie blocking are therefore insufficient; fingerprinting resistance requires browser-level protections, and even those are probabilistic.

— End of Report —